

```
> chisq.test(data)
```

Pearson's Chi-squared test

```
data: data
```

```
X-squared = 59.429, df = 42, p-value = 0.0393
```

A p-value of less than 0.05 shows that there is a strong correlation between cylinder and horsepower.

The *chisq.test* function accepts the following arguments:

Arguments	Description
x	A numeric vector or matrix
y	A numeric vector, or ignored by x is a matrix
correct	A logical value on whether to apply continuity correction
p	A vector of probabilities
rescale.p	A logical scalar to sum to 1
simulate.p.value	A logical value to whether compute p-values by Monte Carlo simulation

Beta

The Beta function is mathematically defined by the integral for two inputs, alpha and beta, as shown below:

$$B(\alpha, \beta) = \frac{\Gamma(\alpha)\Gamma(\beta)}{\Gamma(\alpha + \beta)} = \int_0^1 t^{\alpha-1}(1-t)^{\beta-1} dt$$

Figure 4: Beta distribution

You can use the *beta()* function for the P and Q data sets, as follows:

```
> beta(P, Q)
[1] 7.307338e-100 7.307338e-100 2.515827e-100 5.970038e-162
2.168131e-190
[6] 8.246568e-192 1.139582e-245 1.211659e-144 2.186038e-63
1.931142e-65
```

The *lbeta()* function provides the natural logarithm of the beta function, as demonstrated below:

```
> lbeta(P, Q)
[1] -228.2696 -228.2696 -229.3359 -371.2320 -436.7173
-439.9865 -564.0027
[8] -331.3803 -144.2808 -149.0099
```

The *dbeta()* function provides the density distribution. An example is given below with a plot (Figure 5).

```
> p = seq(0, 5, length=50)
> plot(p, dbeta(p, 3, 5), type='l')
```

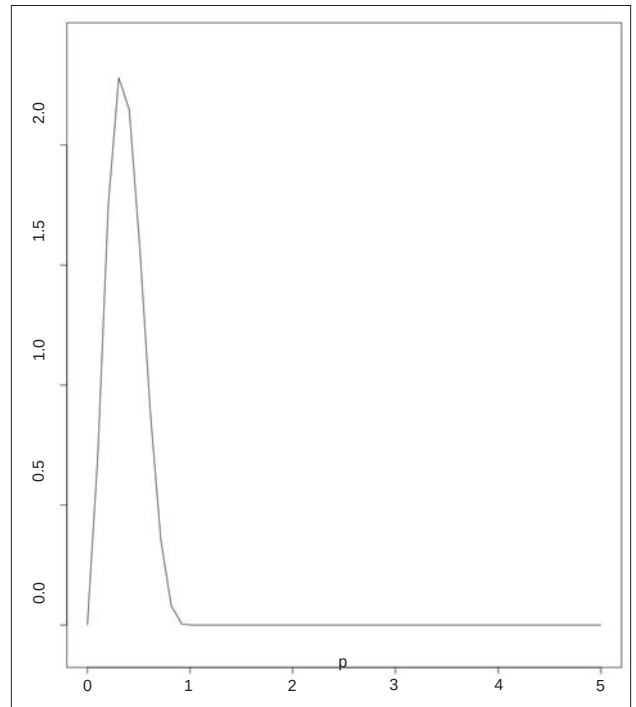


Figure 5: DBeta distribution

Gamma

The Gamma distribution is a continuous probability distribution that has a shape (z) and scale parameter (k), which is defined as shown in Figure 6.

$$\Gamma(\alpha) = \lambda^\alpha \int_0^\infty y^{\alpha-1} e^{-\lambda y} dy$$

Figure 6: Gamma distribution

The *dgamma()* function provides the density function. In the following example, a sequence of values is produced and a plot for its gamma distribution is generated.

```
> a <- seq(0, 50, by=0.01)
> y <- dgamma(a, shape=3)
> plot(y)
```

The *lgamma()* function provides the natural logarithm of the absolute value of the gamma function. For example:

```
> l <- lgamma(a)
> head(l)
[1] Inf 4.599480 3.900805 3.489971 3.197078 2.968879
```